

Processo Seletivo

PET Engenharia de Computação

PET-CHAT

Equipe Roxa - Tema: Redes de Computadores

PROPOSTA

Construir um projeto de Redes de Computadores

Prazo de duas semanas



O QUE É O PET-CHAT?

O PET-CHAT é um chat de terminal: um servidor aceita várias conexões ao mesmo tempo e retransmite as mensagens entre todos os clientes conectados, como uma sala de bate-papo.

protocol.py

network.py

server.py

client_interface.py

MEMBROS E SUBDIVISÃO

ÍCARO

MARÍLIA

ÁLVARO

protocol

network

server

interface

protocol.py é a camada de aplicação: define o que uma mensagem significa: tipo, remetente e conteúdo, sempre em JSON. E também a nossa criptografia ponta a ponta. Se os clientes combinarem uma chave de sala, o conteúdo é cifrado antes de sair do cliente.

Esse arquivo não sabe nada sobre sockets — só sobre o formato e o significado da mensagem.

Tipos de mensagem:

- ENTRAR
- SAIR
- TEXTO





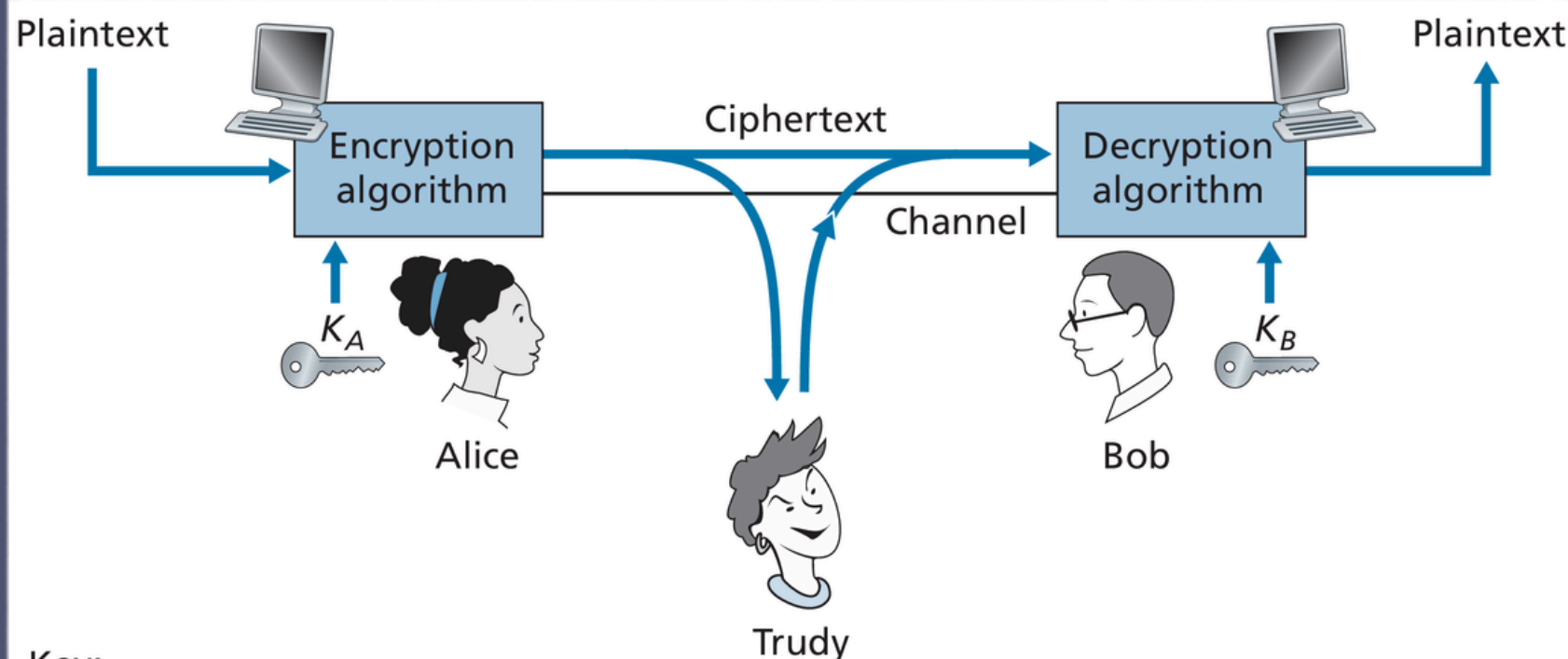
protocol

Validação Prática e Fundamentação Teórica



Referência Bibliográfica

Kurose & Ross — Redes de Computadores e a Internet: Uma Abordagem Top-Down (Capítulo 8: Segurança em Redes)



Inspeção de Tráfego com Wireshark

1

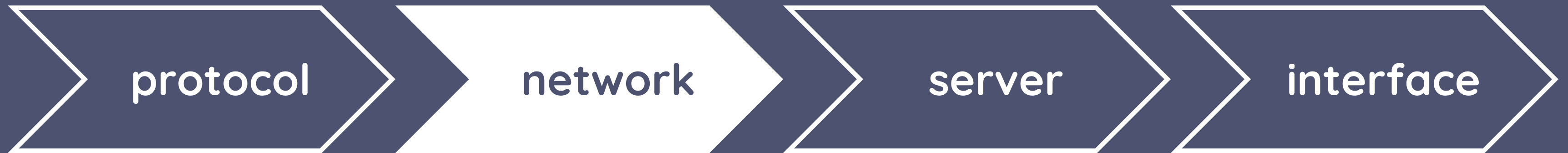
O TCP (Camada de Transporte) garante a entrega confiável dos segmentos, mas trafega dados em texto claro por padrão.

2

Com a cifra E2EE implementada no protocol.py (Camada de Aplicação), o sniffer captura apenas bytes cifrados em Base64.



O servidor atua como roteador cego: confidencialidade garantida de ponta a ponta.



O **network.py** é responsável pela criação e manejo dos sockets. É nesse arquivo que os eles são criados e aplicados. As funções que utilizam esse mecanismo foram definidas : criar o socket, enviar e receber mensagens.

criarsocket()

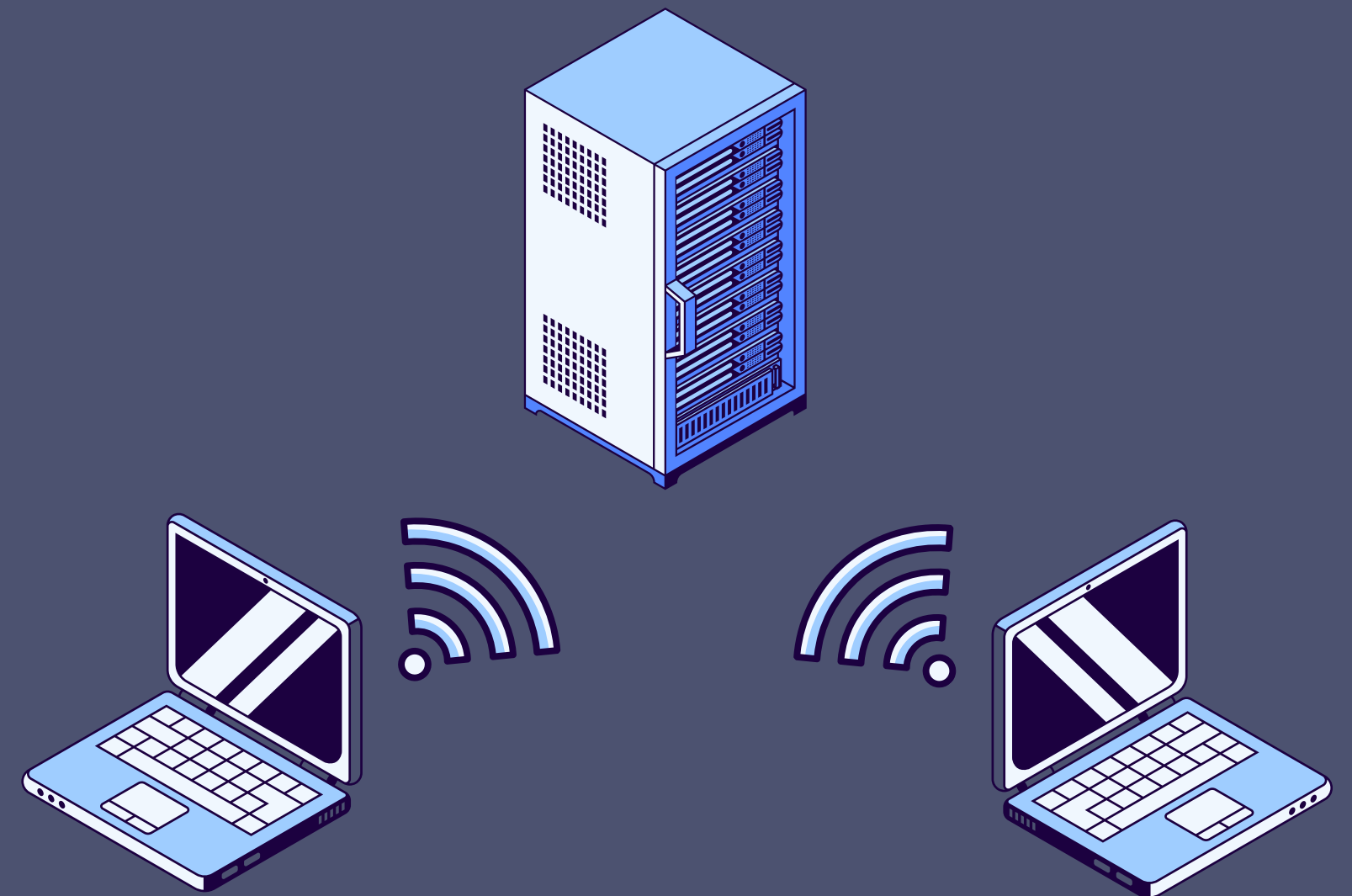
A função retorna um socket, com protocolo TCP e endereço do tipo IPv4.

O network não usa nenhum dos outros arquivos; ele único com conhecimento dos sockets, e o único que os utiliza diretamente.





É aqui que as duas peças se encontram pela primeira vez: o **server.py** usa network para aceitar conexões e trocar bytes, e protocol para entender o que cada mensagem significa. Cada cliente tem sua própria thread, e toda mensagem recebida é retransmitida para os demais, inclusive avisando entradas e saídas, mesmo que repentinas.





O **client_interface.py** é o arquivo que roda o código principal, ele junta todas as peças do sistema: usa network para conectar e trocar bytes com o servidor, e protocol para montar e decifrar as mensagens. Uma thread separada fica só ouvindo o que chega, enquanto a principal cuida do que o usuário digita. Assim dá pra receber mensagens sem travar a digitação.

The image shows two screenshots of a terminal window. The top screenshot shows the chat application running with the user 'Matias' connected. The bottom screenshot shows the chat application running with the user 'Icaro' connected. Both screenshots show the chat history and the current chat session.

```
Prompt de Comando C:\Users\Icaro\AppData\Local\...
chat pelo terminal | PET Eng. Comp. UFC

Seu apelido: Matias
Chave da sala: 123
Conectado em 127.0.0.1:5000.
Comandos: /sair | /limpar | /privado <nome> <msg> | /chave <nova_chave> | /salvar
Matias: oii
Icaro: oii
Matias: oii
[PRIVADO de Icaro]: iae
Matias:

C:\Users\Icaro\AppData\Local\...
chat pelo terminal | PET Eng. Comp. UFC

Seu apelido: Icaro
Chave da sala: 123
Conectado em 127.0.0.1:5000.
Comandos: /sair | /limpar | /privado <nome> <msg> | /chave <nova_chave> | /salvar
Icaro: oi
Icaro: ooi
* Matias entrou
Matias: oii
Icaro: oii
Matias: oii
Icaro: /salvar
* Histórico salvo com sucesso em 'historico_chat.txt' (6 mensagens)
Icaro: /privado Matias iae
[PRIVADO para Matias]: iae
Icaro: |
```

Dificuldades sentidas com cada parte do projeto

protocol.py

Abstração das classes;
Formatação da mensagem para envio;
Criptografia nativa (sem libs externas);
Extensibilidade retrocompatível.

network.py

Dificuldade conceitual;
Entendimento dos elementos como
socket, conexão, accept;
Conversão entre strings e bytes.

server.py

Compreensão e manejo das múltiplas
conexões simultâneas;
Protocolos de saída e encerramento de
conexão;

client_interface.py

Coordenar as threads (ouvir x falar);
Integrar todas partes do sistema numa
interface só;
Tratamento dos erros de conexão.

Processo Seletivo

PET Engenharia de Computação

OBRIGADO!

Equipe Roxa - Tema: Redes de Computadores